

# VERITAS

Verified Execution Boundary for Autonomous Agents

## Plano End-to-End de Construção do MVP

Regras, arquitetura, componentes, portabilidade multi-cloud/local e exigências

**Autor:** Crizan Belem Ribeiro

AI Solutions Architect · Estudante de Física

Versão 1.0 — Agosto de 2026

Documento de concepção técnica para validação. Status: pré-prior-art. Nenhuma novidade ou patenteabilidade é assumida.

# Sumário

1. Objetivo, escopo e leitores
  2. Tese, pergunta de pesquisa e propriedades formais
  3. Regras principais do sistema (R1–R14)
  4. Arquitetura lógica e fluxo de execução
  5. Componentes do MVP
  6. Modelo de dados
  7. Protocolo de concorrência e commit
  8. Arquitetura de implantação: local, single-cloud e multi-cloud
  9. Exigências (funcionais, não funcionais, segurança, compliance, operação)
  10. Plano end-to-end por sprints
  11. Estratégia de verificação e testes
  12. Métricas e critérios de aceite
  13. Riscos e mitigações
  14. Fora de escopo do MVP
  15. Decisão de IP e open source
- Referências

# 1. Objetivo, escopo e leitores

Este documento descreve, de ponta a ponta, como construir o MVP do VERITAS: uma fronteira de execução verificada entre agentes autônomos e ferramentas consequentes. O MVP deve ser executável por uma pessoa, reproduzível com um comando, portátil entre execução local, uma única nuvem (AWS, Azure ou GCP) e operação multi-cloud federada, e deve produzir três ativos: uma biblioteca Python instalável, um benchmark público (VERITAS-Bench) e evidência experimental publicável.

O MVP não é um AgentOS, não é um gateway genérico e não é um produto de identidade. Ele responde a uma única pergunta que nenhum sistema disponível em 2026 responde: como preservar invariantes globais de segurança quando múltiplos agentes executam ações localmente autorizadas, concorrentes e individualmente inocentes.

## 1.1 Leitores

- Engenharia: para implementar componentes e adaptadores na ordem definida.
- Segurança/IAM: para avaliar o modelo de ameaças, as hipóteses de confiança e a interface com identidade upstream.
- Pesquisa: para reproduzir o benchmark e verificar as propriedades formais declaradas.
- Plataforma/Cloud: para decidir o modo de implantação e os adaptadores necessários.

## 1.2 Princípios de projeto

- **Verificador, não filtro.** A fronteira prova admissibilidade de trajetórias, não apenas de chamadas isoladas.
- **Solver fora do caminho quente.** Z3/SMT opera em CI e na compilação de políticas; o runtime usa artefatos compilados e certificados verificáveis em  $O(1)$ .
- **Ports & Adapters obrigatórios.** Nenhum componente de domínio importa SDK de nuvem. Toda dependência externa é uma porta com no mínimo um adaptador local.
- **Identidade é consumida, não emitida.** O VERITAS aceita identidade e delegação assinadas por emissores upstream (OIDC/SPIFFE/OAuth token exchange).
- **Fail-closed.** Ausência de identidade, política, calibração ou certificado resulta em negação.
- **Tudo é reproduzível.** Traces endereçados por conteúdo, seeds fixas, versões de modelo e política registradas.

# 2. Tese, pergunta de pesquisa e propriedades formais

**Tese.** *A segurança de agentes é uma propriedade de trajetórias sob incerteza e concorrência, não de tool calls individuais.*

**Pergunta de pesquisa.** *Trajетórias consequentes de agentes podem permanecer verificavelmente admissíveis sob composição, concorrência, incerteza e desvio adversarial de distribuição, mantendo sobrecarga de execução baixa? E qual é o trade-off fundamental entre coordenação e autonomia na manutenção de invariantes globais?*

## 2.1 Álgebra de recursos de autonomia

Uma trajetória  $\tau = (a_1, \dots, a_n)$  consome recursos de um vetor  $R$  definido sobre um **monoide ordenado**. Só entram na álgebra dimensões que satisfazem os axiomas de consumo monotônico: dinheiro, contagem de chamadas, volume de dados exportados e **perda esperada** ( $\sum p_i \cdot L_i \leq B$ ). Risco como probabilidade bruta e profundidade de delegação não são recursos; pertencem às Classes II e III.

$R_{t+1} = R_t - c(a_t)$       #  $c(a)$ : vetor de consumo da ação  
 Admissível( $\tau$ )  $\Leftrightarrow R_t \in S$  para todo  $t$       #  $S$ : região segura (hiper-retângulo no MVP)

## 2.2 Classes de invariantes

| Classe          | Natureza              | Exemplos                                                                                                    | Motor de runtime                | MVP             |
|-----------------|-----------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------|-----------------|
| I — Recursos    | Aditivos, monotônicos | $\Sigma \text{ amount} \leq 10.000 / 24\text{h}$ por destino; $\text{count(API)} \leq 1.000/\text{h}$       | Residual assinado na capability | Sim             |
| II — Temporais  | Ordem e precedência   | $\text{Approval} < \text{Transfer}$ ; $\text{SensitiveRead} \Rightarrow \neg \text{ExternalSend}$ na sessão | Autômato por (sessão, recurso)  | Ciclo 2         |
| III — Separação | Relacionais           | $\text{Initiator} \neq \text{Approver}$ ; $\text{depth(delegation)} \leq 3$                                 | Cedar com contexto de cadeia    | Parcial (Cedar) |

## 2.3 Propriedades formais

**Trajectory Safety (refinamento).** Sob as hipóteses H1-H6, a implementação (tabelas compiladas + capabilities consumíveis + protocolo Prepare/Verify/Commit) refina a especificação: toda trajetória que a implementação autoriza satisfaz todos os invariantes suportados. O enunciado não é tautológico porque o objeto provado é a implementação, não a definição.

**Progress (empírico).** No domínio suportado, trajetórias seguras são autorizadas com probabilidade próxima de 1; mede-se Security  $\times$  Autonomy  $\times$  Latency, nunca só bloqueios.

**No Double-Spend.** Para todo recurso  $r$  e instante  $t$ , a soma dos residuais de capabilities vivas de  $r$  nunca excede  $R_{r,t}$ .

### Hipóteses explícitas (cada uma é também um caso de ataque no bench)

- H1 — O emissor de capabilities é honesto e sua chave privada não está comprometida.
- H2 — Assinaturas (Ed25519) são inforjáveis no modelo computacional padrão.
- H3 — Toda ferramenta protegida verifica o certificado antes de executar (tool boundary cooperativa).
- H4 — Desvio de relógio entre emissor e ferramenta é limitado por  $\delta$  conhecido.
- H5 — Identidade e delegação upstream são autênticas (emissor OIDC/SPIFFE confiável).
- H6 — Nenhuma capability sobrevive ao seu commit, compensação ou expiração.

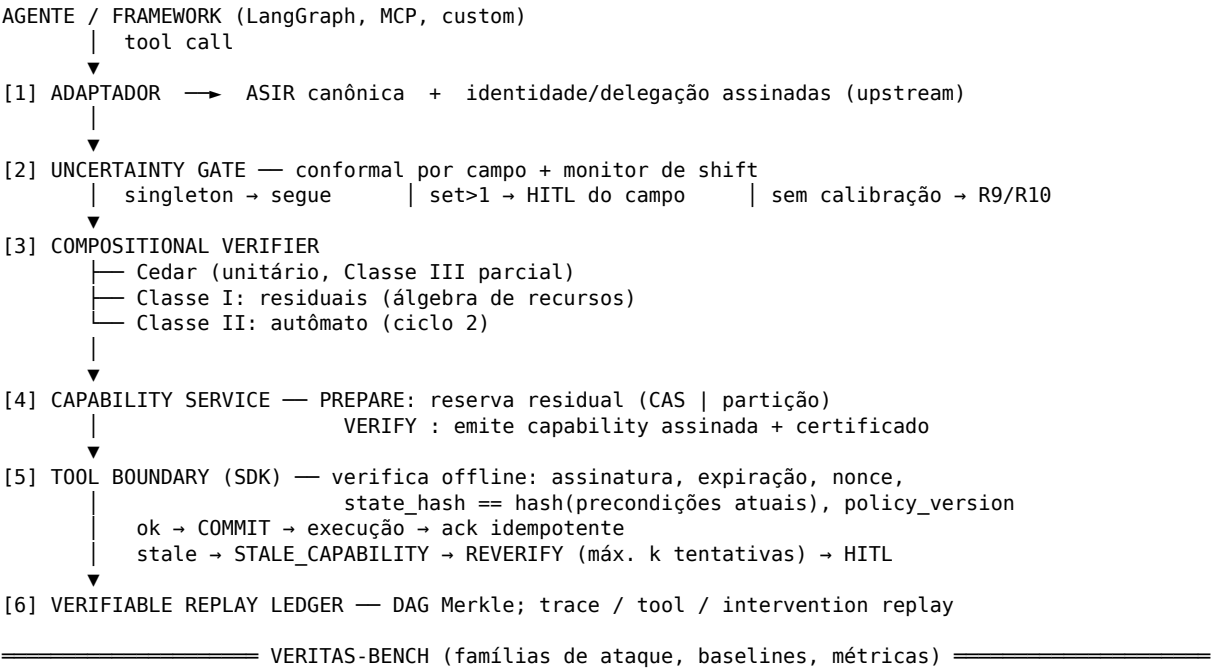
## 3. Regras principais do sistema

As regras abaixo são invariantes de projeto. Violar qualquer uma delas invalida as propriedades da Seção 2 e deve falhar em CI.

| ID | Regra                                                                                                                                                | Justificativa                                            |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| R1 | Nenhuma ferramenta protegida executa sem capability válida, não expirada, não consumida e com certificado verificável offline.                       | Fronteira de execução real, não aconselhamento ao LLM.   |
| R2 | Capabilities são consumíveis: uma vez usadas em commit, geram sucessora com residual reduzido e jamais são reutilizáveis (nonce + índice de cadeia). | Elimina reuso e replay.                                  |
| R3 | Residuais de um mesmo recurso só são emitidos por CAS linearizável ou por partição prévia. Nunca por leitura-então-escrita não atômica.              | Propriedade No Double-Spend.                             |
| R4 | Toda capability é vinculada a: hash da ASIR, hash de precondições de estado, versão da política, expiração, nonce e identidade upstream.             | State-bound authorization; elimina TOCTOU e policy race. |

| ID  | Regra                                                                                                                                                                          | Justificativa                                       |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| R5  | Mudança de versão de política invalida capabilities emitidas sob a versão anterior ainda não commitadas.                                                                       | Policy race.                                        |
| R6  | Aprovação humana é uma assinatura sobre Hash(ASIR canônica) com nonce e expiração; a renderização exibida é derivada deterministicamente da mesma ASIR.                        | Approval mutation; WYSIWYS.                         |
| R7  | O solver SMT nunca está no caminho de runtime. Runtime usa tabelas compiladas e aritmética de residuais.                                                                       | p95 do caminho local < 5 ms.                        |
| R8  | Toda decisão (ALLOW/DENY/REQUIRE_APPROVAL/STALE) é persistida no ledger com hash encadeado antes de ser comunicada.                                                            | Tamper-evidence e replay.                           |
| R9  | O Uncertainty Gate só emite garantia estatística quando existe massa de calibração $\geq 1/\alpha$ para o tipo de ação; caso contrário, a ação é HITL obrigatório ou proibida. | Evita $\alpha$ incalibrável.                        |
| R10 | Ações de alto risco ( $\alpha$ alvo < 0,01) nunca dependem do gate estatístico; são decididas por política determinística ou HITL.                                             | Fronteira limpa entre estatístico e determinístico. |
| R11 | Commit falho ou timeout dispara compensação com prazo; o residual só retorna após confirmação idempotente de não-execução.                                                     | Evita vazamento e replay de orçamento.              |
| R12 | Código de domínio não importa SDKs de nuvem. Toda dependência externa passa por uma porta com adaptador local de referência.                                                   | Portabilidade e testabilidade.                      |
| R13 | Ausência de identidade, política, calibração, chave ou conectividade com o store resulta em DENY.                                                                              | Fail-closed.                                        |
| R14 | Nenhum segredo ou credencial de ferramenta é exposto ao agente ou ao contexto do LLM. A ferramenta recebe apenas a capability.                                                 | Zero standing privilege.                            |

## 4. Arquitetura lógica e fluxo de execução



### 4.1 Fluxo Prepare → Verify → Commit

| Fase       | Quem                          | O que acontece                                                                                                 | Saída                                   |
|------------|-------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Prepare    | Capability Service            | Reserva residual do recurso (CAS sobre o store ou consumo da partição do agente). Registra intenção no ledger. | Reserva com TTL curto                   |
| Verify     | Verifier + Capability Service | Avalia Cedar + invariantes compilados; calcula state_hash sobre precondições declaradas; assina capability.    | Capability + certificado                |
| Commit     | Tool Boundary                 | Reverifica certificado offline, compara state_hash, executa, grava ack idempotente.                            | Execução + ack no ledger                |
| Compensate | Capability Service            | Se não houver ack até o prazo, confirma não-execução e devolve residual (R11).                                 | Residual restaurado ou perda registrada |

## 5. Componentes do MVP

### 5.1 ASIR — Agent Safety Intermediate Representation

Contrato central da biblioteca. Schema Pydantic canônico, serialização determinística (JSON canônico RFC 8785) para que Hash(ASIR) seja estável entre linguagens e nuvens.

- Entradas: tool call bruta do framework + token de identidade upstream.
- Saídas: ASIR validada, normalizada e canônica; hash SHA-256.
- Adaptadores no MVP: LangGraph (obrigatório), MCP (desejável), custom HTTP.
- Não faz: decisão de política, inferência de intenção, classificação de PII por LLM (apenas rótulos estruturais).

### 5.2 Uncertainty Gate

Predição conformal split por campo da ASIR com escores de não-conformidade calibrados em holdout rotulado;  $\alpha$  por classe de risco (R9/R10); monitor de shift por razão de densidade sobre embeddings de contexto.

- Entradas: ASIR, distribuição de saída do modelo (logprobs ou amostras), conjunto de calibração.
- Saídas: conjunto de predição por campo, tamanho do conjunto, flag de shift, decisão (segue / HITL de campo / sem garantia).
- Métricas próprias: Coverage\_clean, Coverage\_shift, Coverage\_injection,  $\Delta$ Coverage, AbstentionRate,  $E[|C(X)|]$ , HumanReviewCost.
- No MVP: ciclo 2. Ciclo 1 usa gate determinístico (bypass com registro) para não bloquear a tese de concorrência.

### 5.3 Compositional Verifier

Compila políticas Cedar + DSL de invariantes em (a) fórmulas SMT para CI e (b) tabelas de decisão + parâmetros de residual para runtime.

- CI: prova consistência entre políticas, contenção  $\text{scope}(\text{capability}) \subseteq \text{allow}(\text{policy})$ , inalcançabilidade de estados proibidos até profundidade k (bounded model checking). SAT  $\Rightarrow$  contraexemplo anexado ao PR.
- Runtime: lookup em tabela + aritmética de residual; sem solver (R7).
- DSL mínima: invariant money:  $\text{sum}(\text{transfer.amount}) \text{ over } 24\text{h per destination} \leq 10000$ .
- Dependências: cedar-policy (Python bindings), z3-solver, hypothesis.

## 5.4 Capability Service

Emite capabilities consumíveis (PASETO v4 público, Ed25519) com residual, state\_hash, policy\_version, nonce, índice de cadeia e certificado compacto de contenção.

- Modos de reserva: CAS linearizável (porta StateStore) ou partição prévia por agente (porta BudgetPartitioner). Híbrido: partição por default, CAS sob contenção.
- Compensação com prazo e chave de idempotência (R11).
- Chaves via porta Signer (KMS local/AWS KMS/Azure Key Vault/GCP KMS).
- Nunca vê segredos das ferramentas (R14).

## 5.5 Tool Boundary SDK

Middleware leve que a ferramenta protegida (ou um proxy à frente dela) executa para verificar certificados offline e cumprir o commit.

- Verificação: assinatura, expiração, nonce não visto, policy\_version vigente, state\_hash sobre condições que a ferramenta sabe calcular.
- Para ferramentas não cooperativas: proxy VERITAS com janela de expiração curta e condições reduzidas (degradação documentada).
- Emite ack idempotente ao ledger.

## 5.6 Verifiable Replay Ledger

DAG Merkle de nós endereçados por conteúdo:  $H_i = \text{Hash}(H_{\text{parents}}, \text{payload}_i)$ . Append-only.

- Três replays: trace (reprodução exata, sem modelo), tool (ferramentas mockadas com saídas gravadas), intervention (substitui um nó e propaga).
- API: `replay(trace_id, intervene={node: value}, mode=...)`.
- Persistência via porta LedgerStore (SQLite local / Postgres / DynamoDB / Cosmos / Firestore).
- Claim defensável: dentro do ambiente reexecutável capturado, a intervenção altera a trajetória desta forma. Não afirma causalidade geral.

## 5.7 VERITAS-Bench

Harness de avaliação com famílias de ataque, baselines e métricas. É o ativo de pesquisa e pode valer mais do que a biblioteca.

- Famílias: atomic, fractionation, temporal evasion, parallel double-spend, delegation laundering, approval mutation, cross-tool composition, policy race, clock skew, capability replay, compensation abuse.
- Baselines: sem proteção; Cedar unitário (Strands/AWS cedar-for-agents); OPA; proxy com token OAuth atenuado. Qualquer baseline adicional só entra com DOI/repositório verificado.
- Ambientes determinísticos: pagamentos, banco de dados, mensageria — com dois ou mais agentes concorrentes.

# 6. Modelo de dados

## 6.1 ASIR canônica

```
{
  "asir_version": "1.0",
  "agent_id": "finance-agent-01",
  "principal": {"sub": "user:alice", "iss": "https://idp.example", "act": ["finance-agent-01"]},
  "delegation": ["user:alice", "orchestrator-07", "finance-agent-01"],
  "action": "payment.transfer",
  "resource": "account-123",
  "parameters": {"amount": 900, "currency": "BRL", "destination": "acct-987"},
  "purpose": "invoice-payment",
}
```

```

"labels": {"data_sensitivity": "financial", "irreversible": true},
"context": {"session_id": "s-42", "request_ts": "2026-08-21T12:00:00Z", "source_observations": ["obs:sha256:..."]}
}

```

## 6.2 Capability consumível

```

{
  "cap_id": "cap:sha256:...", "chain_index": 12, "parent_cap": "cap:sha256:...",
  "asir_hash": "sha256:...", "state_hash": "sha256:...", # hash das precondições declaradas
  "preconditions": {"account-123.balance_ge": 900, "policy_version": "v7"},
  "residual": {"money:acct-987:24h": 9100, "api:payment:1h": 41},
  "policy_version": "v7", "issued_at": "...", "expires_at": "...", # TTL curto (s)
  "nonce": "...", "issuer_kid": "kms:key-2026-08",
  "certificate": {"claim": "scope ≤ allow(v7)", "proof_digest": "sha256:..."},
  "sig": "ed25519:..."
}

```

## 6.3 Nó do ledger

```

Node {
  id          : Hash(parents u payload)
  type        : OBSERVATION | ASIR | GATE_DECISION | VERIFIER_DECISION | CAPABILITY |
               HUMAN_APPROVAL | TOOL_INPUT | TOOL_OUTPUT | COMMIT_ACK | COMPENSATION
  payload     : bytes (canônico)  model_id, model_output_hash, seed (quando aplicável)
  policy_version, verifier_result, parents[], ts
}

```

# 7. Protocolo de concorrência e commit

Este é o núcleo científico do MVP. Encadear capabilities (estilo UTXO) não impede gasto duplo por si só: dois agentes podem ler o mesmo residual e derivar sucessoras concorrentes. A serialização precisa vir de um dos mecanismos abaixo, e a curva de trade-off entre eles é o resultado principal a ser medido.

| Mecanismo        | Como funciona                                                                            | Garantia                                   | Custo                                                                               | Quando usar                       |
|------------------|------------------------------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------|
| CAS linearizável | Reserva por compare-and-swap no StateStore (chave = recurso). Uma única escrita vence.   | No Double-Spend exato                      | Ponto de serialização por recurso; latência sob contenção; disponibilidade do store | Recursos escassos e de alto valor |
| Partição prévia  | O emissor divide R em fatias por agente antes da execução; cada agente consome só a sua. | No Double-Spend sem coordenação em runtime | Fragmentação; pode negar ação viável no agregado                                    | Muitos agentes, baixa contenção   |
| Híbrido (MVP)    | Partição por default; CAS ao esgotar a fatia; rebalanceamento assíncrono off-path.       | Exato, com autonomia parcial               | Complexidade de rebalanceamento                                                     | Default do VERITAS                |

## 7.1 Falhas e compensação

- Commit sem ack até  $T_{ack}$ : o serviço consulta a ferramenta pela chave de idempotência; se não executou, devolve o residual; se executou, registra o ack tardio; se indeterminado, marca perda e escalona.
- STALE\_CAPABILITY: no máximo  $k = 3$  reverificações; depois, HITL. Evita livelock sob churn de estado.
- Clock skew acima de  $\delta$ : capability tratada como expirada (fail-closed).

## 7.2 Parâmetros a medir no bench

- Throughput (ações/s) e p50/p95/p99 para concorrência 1, 10, 100, 1000, em cada mecanismo.
- Taxa de negação de ações viáveis no agregado (custo da partição).



- Taxa de contenção e tempo médio de espera no CAS.
- Curva coordenação × autonomia: fração de decisões que exigiram serialização versus fração de trajetórias seguras autorizadas.

## 8. Arquitetura de implantação: local, single-cloud e multi-cloud

Toda dependência de infraestrutura é uma **porta** no domínio e um **adaptador** por ambiente. O mesmo binário roda em qualquer modo trocando apenas configuração (variáveis de ambiente ou arquivo `veritas.toml`). O adaptador local é a implementação de referência e o oráculo dos testes de conformidade dos demais.

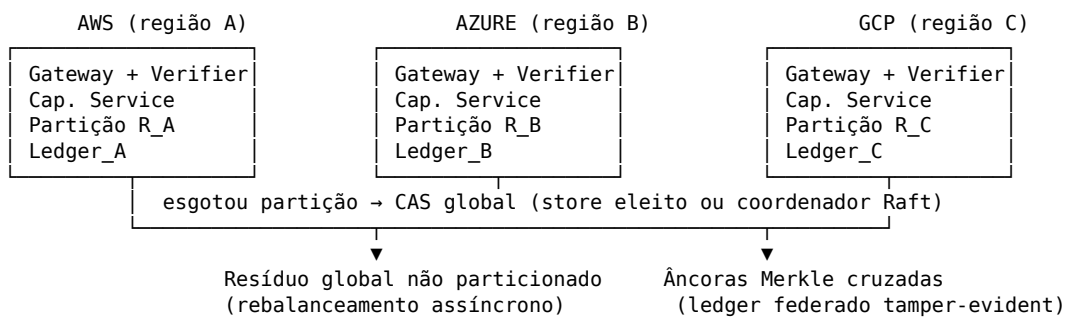
### 8.1 Portas e adaptadores

| Porta            | Responsabilidade                                          | Local                                  | AWS                                   | Azure                         | GCP                                |
|------------------|-----------------------------------------------------------|----------------------------------------|---------------------------------------|-------------------------------|------------------------------------|
| StateStore (CAS) | Reservas de residual com escrita condicional linearizável | SQLite (BEGIN IMMEDIATE) / Redis local | DynamoDB (ConditionExpression)        | Cosmos DB (ETag / optimistic) | Firestore (transactions) / Spanner |
| LedgerStore      | Append-only, consulta por hash e por trace                | SQLite / Postgres                      | DynamoDB + S3 Object Lock             | Cosmos + Blob immutable       | Firestore + GCS retention          |
| Signer           | Assinatura Ed25519 e rotação de chaves                    | Chave em arquivo (dev) / SoftHSM       | AWS KMS                               | Azure Key Vault               | Cloud KMS                          |
| SecretStore      | Segredos das ferramentas (nunca expostos ao agente)       | .env cifrado / Vault dev               | Secrets Manager                       | Key Vault                     | Secret Manager                     |
| IdentityVerifier | Validação de tokens OIDC/SPIFFE e delegação (act)         | Emissor OIDC local (mock) / SPIRE      | Cognito / IAM Identity Center / SPIRE | Entra ID                      | Cloud Identity / Workload Identity |
| EventBus         | Compensação assíncrona, rebalanceamento                   | Fila em memória / Redis Streams        | SQS + EventBridge                     | Service Bus                   | Pub/Sub                            |
| Telemetry        | Traços, métricas, logs                                    | OTel Collector + Prometheus + Grafana  | OTel → CloudWatch / X-Ray             | OTel → Azure Monitor          | OTel → Cloud Trace / Monitoring    |
| Compute          | Execução do gateway e do serviço de capabilities          | Docker Compose                         | ECS Fargate / Lambda                  | Container Apps / AKS          | Cloud Run / GKE                    |
| PolicyStore      | Versões de política compiladas e assinadas                | Git + filesystem                       | S3 versionado                         | Blob versionado               | GCS versionado                     |
| ModelProvider    | Modelo do agente e logprobs para o gate                   | Ollama / LLM simulado determinístico   | Bedrock                               | Azure OpenAI                  | Vertex AI                          |

### 8.2 Modos de implantação

| Modo                 | Descrição                                                                                   | Coordenação de residuais                                                                                                                                                    | Requisito específico                                                                                                                            |
|----------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Local                | Docker Compose com todos os adaptadores locais. Modo padrão de desenvolvimento, CI e bench. | CAS em SQLite/Redis; partição em memória                                                                                                                                    | make -demo sobe tudo com um comando                                                                                                             |
| Single-cloud         | Uma nuvem, adaptadores nativos dela. Identidade via IdP da própria nuvem.                   | CAS no store nativo; partição por agente                                                                                                                                    | KMS da nuvem para assinatura; política versionada em object storage                                                                             |
| Multi-cloud isolado  | Uma instância independente por nuvem, sem recurso compartilhado entre elas.                 | Independente por nuvem                                                                                                                                                      | Namespaces de recurso disjuntos; ledger por nuvem                                                                                               |
| Multi-cloud federado | Agentes em nuvens distintas consomem o mesmo recurso lógico (ex.: orçamento global).        | Partição entre nuvens como default; CAS global apenas para o resíduo não particionado, em um store eleito (Spanner / DynamoDB Global Tables) ou via coordenador leve (Raft) | Relógios com 6 limitado; ledgers federados por cross-anchoring de raízes Merkle; chave de verificação pública distribuída a todas as boundaries |

### 8.3 Topologia federada



### 8.4 Regras de portabilidade

- Testes de conformidade de porta: a mesma suíte roda contra cada adaptador; o adaptador local é o oráculo.
- Nenhum `import boto3 / azure.* / google.cloud.*` fora de `veritas/adapters/`; lint em CI bloqueia.
- Infraestrutura como código por nuvem (Terraform) com módulos equivalentes; variáveis de ambiente idênticas entre modos.
- Política compilada é um artefato assinado, idêntico em todas as nuvens; a versão vigente é propagada pelo PolicyStore e verificada na boundary (R5).

## 9. Exigências

### 9.1 Funcionais

| ID   | Exigência                                                                                                  | Prioridade  |
|------|------------------------------------------------------------------------------------------------------------|-------------|
| RF01 | Interceptar tool calls de LangGraph e normalizar em ASIR canônica com hash estável.                        | Obrigatória |
| RF02 | Compilar Cedar + DSL de invariantes em tabelas de runtime e fórmulas SMT de CI.                            | Obrigatória |
| RF03 | Emitir capabilities consumíveis com residual, state_hash, policy_version, nonce, certificado e assinatura. | Obrigatória |

| ID   | Exigência                                                                                    | Prioridade  |
|------|----------------------------------------------------------------------------------------------|-------------|
| RF04 | Reservar residuais por CAS e por partição; operar em modo híbrido.                           | Obrigatória |
| RF05 | Verificar capabilities offline na boundary e executar Prepare/Verify/Commit com compensação. | Obrigatória |
| RF06 | Registrar todo evento em ledger Merkle e suportar trace, tool e intervention replay.         | Obrigatória |
| RF07 | Aprovação humana assinada sobre Hash(ASIR) com renderização determinística.                  | Obrigatória |
| RF08 | Executar VERITAS-Bench com todas as famílias de ataque e ao menos dois baselines.            | Obrigatória |
| RF09 | Uncertainty Gate conformal por campo com $\alpha$ por classe de risco e monitor de shift.    | Ciclo 2     |
| RF10 | Invariantes temporais (Classe II) por autômato.                                              | Ciclo 2     |
| RF11 | Adaptador MCP.                                                                               | Desejável   |

## 9.2 Não funcionais

| ID    | Exigência                                                            | Alvo                                                                            |
|-------|----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| RNF01 | Latência do caminho local de política (sem rede e sem ferramenta)    | p95 < 5 ms; p99 < 10 ms                                                         |
| RNF02 | Verificação offline de capability na boundary                        | p95 < 1 ms                                                                      |
| RNF03 | Throughput sob concorrência 1/10/100/1000                            | Medido e publicado por mecanismo                                                |
| RNF04 | Disponibilidade do modo partição sob indisponibilidade do CAS global | Continua autorizando dentro da fatia local                                      |
| RNF05 | Reprodutibilidade                                                    | Um comando sobe demo + bench; seeds fixas; resultados idênticos entre execuções |
| RNF06 | Portabilidade                                                        | Mesma suíte de conformidade verde em local, AWS, Azure e GCP                    |
| RNF07 | Observabilidade                                                      | OTel em 100% das decisões; métricas por componente                              |
| RNF08 | Tamanho do certificado na capability                                 | < 1 KB                                                                          |

## 9.3 Segurança

- Chaves de assinatura em KMS/HSM nos modos cloud; rotação com kid e período de sobreposição.
- Nonces únicos com janela de memória  $\geq$  TTL máximo de capability; verificação de não-reuso na boundary.
- Segredos de ferramentas nunca transitam pelo agente, pelo contexto do LLM ou pelo ledger (R14).
- Modelo de ameaças documentado: agente comprometido por injeção, agente colusivo, orquestrador malicioso, operador que altera política, relógio manipulado, boundary que ignora certificado.
- Dependências fixadas por hash; SBOM gerado em CI; imagens assinadas (cosign).

## 9.4 Compliance e auditoria

- Ledger append-only com raízes Merkle ancoradas periodicamente em armazenamento imutável (Object Lock / immutable blob / retention).

- Cada decisão contém identidade, delegação, ação, política e razão — alinhado a requisitos de registro para sistemas de alto risco (EU AI Act) e aos princípios de identificação, autorização, auditoria e não-repúdio do NIST para agentes.
- Dados pessoais em ASIR minimizados e rotulados; ledger não armazena payloads sensíveis em claro (hash + referência cifrada).

## 9.5 Operação e desenvolvimento

- Ambiente alvo: Windows + Docker Desktop + VSCode; Python 3.12+; uv ou poetry; pytest; hypothesis; ruff; mypy estrito.
- CI (GitHub Actions): lint de portabilidade, testes unitários, property-based, prova SMT de políticas, conformidade de adaptadores (local sempre; cloud em jobs opcionais com credenciais), bench reduzido.
- Documentação: README, ARCHITECTURE, THREAT\_MODEL, BENCH, ADRs numerados.
- Licença decidida apenas ao final do Sprint 0 e revisitada no Sprint 6.

## 10. Plano end-to-end por sprints

Sprints de três a quatro semanas, executáveis por uma pessoa. Cada sprint tem critério de saída objetivo; sem saída, o próximo não começa.

| Sprint                                | Entregáveis                                                                                                                                                                                                                                     | Critério de saída                                                                                                                                |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 — Prior art e fundação (2 sem.)     | Revisão de patentes e literatura sobre proof-carrying authorization, consumable capabilities, conformal para tool-use, trace replay; matriz de novidade; nome definitivo; esqueleto do repo com portas e adaptadores locais; ADR-001 a ADR-005. | Declaração escrita do que o VERITAS faz que nenhum sistema encontrado faz; repo compila com <code>make test</code> .                             |
| 1 — ASIR + Verifier (Classe I)        | Schema canônico, hash estável; DSL de invariantes; compilação para tabelas e SMT; provas de consistência e contenção em CI; property-based com Hypothesis.                                                                                      | Z3 encontra contraexemplo em política propositalmente inconsistente e retorna UNSAT na correta; cenário de fracionamento negado em teste.        |
| 2 — Capability Service + concorrência | Capabilities consumíveis assinadas; CAS sobre SQLite/Redis; partição; híbrido; compensação; testes de corrida com 2...1000 agentes simulados.                                                                                                   | Zero double-spend em $10^6$ tentativas concorrentes por mecanismo; curva latência $\times$ concorrência medida.                                  |
| 3 — Tool Boundary + Ledger            | SDK de verificação offline; Prepare/Verify/Commit; STALE/REVERIFY; ledger Merkle; trace, tool e intervention replay; aprovação assinada.                                                                                                        | Hero scenario fim a fim reproduzido localmente: doze transferências de 900 negadas na décima segunda; replay aponta o nó de observação injetado. |
| 4 — Adaptadores cloud                 | Adaptadores AWS, Azure e GCP para todas as portas; Terraform por nuvem; suíte de conformidade; modo federado com partição + CAS global.                                                                                                         | Suíte de conformidade verde em quatro ambientes; demo federado com orçamento global compartilhado entre duas nuvens.                             |
| 5 — VERITAS-Bench + gate              | Todas as famílias de ataque; baselines; métricas de segurança, autonomia e latência; Uncertainty Gate conformal (ciclo 2) com medição de $\Delta$ Coverage.                                                                                     | Relatório comparativo reproduzível; ao menos três famílias em que os baselines falham e o VERITAS não.                                           |
| 6 — Paper, IP e OSS                   | Artigo de workshop; modelo de ameaças final; decisão de depósito ou publicação; README e licença.                                                                                                                                               | Submissão feita; decisão de IP registrada antes de qualquer repo público.                                                                        |

## 11. Estratégia de verificação e testes

| Nível          | Técnica                                                                     | O que prova                                                                                                                  |
|----------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Unitário       | pytest                                                                      | Comportamento de cada componente e adaptador.                                                                                |
| Property-based | Hypothesis gera milhares de trajetórias aleatórias                          | Nenhuma trajetória autorizada viola invariante; propriedades de monoide da álgebra.                                          |
| Formal (CI)    | Z3: $\exists \tau: \text{Authorized}(\tau) \wedge \text{Violation}(\tau)?$  | SAT $\Rightarrow$ bug de política com contraexemplo; UNSAT $\Rightarrow$ evidência formal dentro do modelo (profundidade k). |
| Concorrência   | Testes de corrida com relógios simulados, partições de rede e falhas de ack | No Double-Spend; compensação correta; ausência de livelock.                                                                  |
| Conformidade   | Mesma suíte contra cada adaptador                                           | Equivalência semântica local $\leftrightarrow$ AWS $\leftrightarrow$ Azure $\leftrightarrow$ GCP.                            |
| Adversarial    | VERITAS-Bench                                                               | Famílias de ataque bloqueadas; baselines comparados; custo em autonomia e latência.                                          |
| Replay         | Reexecução de traces gravados                                               | Determinismo do ledger; intervenções alteram trajetória conforme esperado.                                                   |

## 12. Métricas e critérios de aceite

| Métrica                                 | Definição                                                                                                   | Alvo do MVP                                   |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Ataques composicionais bloqueados       | Fração das famílias de composição/concorrência negadas pelo VERITAS e aceitas pelos baselines unitários     | 100% das famílias definidas                   |
| Double-spend                            | Ocorrências em teste de corrida                                                                             | 0 em $10^6$ tentativas por mecanismo          |
| Safe Action Pass Rate                   | Trajetois seguras autorizadas no domínio suportado                                                          | $\geq 99\%$ (partição pode reduzir; reportar) |
| Inconsistências de política encontradas | Conjuntos reais (Cedar/OPA públicos) em que o verificador de CI encontra contradição ou vazamento de escopo | Reportar número absoluto                      |
| Latência local                          | p50/p95/p99 do caminho de política                                                                          | p95 < 5 ms                                    |
| Verificação na boundary                 | Tempo de verificação offline                                                                                | p95 < 1 ms                                    |
| Curva coordenação x autonomia           | Fração de decisões serializadas vs. fração de trajetórias seguras autorizadas, por mecanismo                | Publicada                                     |
| $\Delta$ Coverage (ciclo 2)             | Coverage_clean – Coverage_attack                                                                            | Medida; sem promessa de valor                 |
| Audit completeness                      | Decisões com identidade, delegação, ação, política, razão e hash encadeado                                  | 100%                                          |
| Reprodutibilidade                       | Demo e bench idênticos entre execuções e ambientes                                                          | Exigida                                       |

## 13. Riscos e mitigações

| Risco                                                                                | Impacto                  | Mitigação                                                                                          |
|--------------------------------------------------------------------------------------|--------------------------|----------------------------------------------------------------------------------------------------|
| Prior art cobre consumable capabilities ou proof-carrying authorization para agentes | Perda do claim principal | Sprint 0 com busca de patentes; pivotar a novidade para o protocolo de concorrência e para o bench |

| Risco                                    | Impacto                           | Mitigação                                                                           |
|------------------------------------------|-----------------------------------|-------------------------------------------------------------------------------------|
| CAS global vira gargalo no modo federado | Latência e disponibilidade        | Partição como default; CAS só para resíduo; medir e publicar a curva                |
| Ferramentas não cooperam com state_hash  | State-bound degrada para TTL      | Proxy VERITAS com precondições reduzidas; documentar como limitação e propor padrão |
| Calibração insuficiente para o gate      | Garantias estatísticas vazias     | R9/R10: sem massa, sem garantia; alto risco sempre determinístico                   |
| Escopo cresce de volta para AgentOS      | MVP não termina                   | Seção 14 como contrato; qualquer adição exige ADR e remoção equivalente             |
| Divergência semântica entre adaptadores  | Comportamento diferente por nuvem | Suíte de conformidade única com adaptador local como oráculo                        |
| Custo de execução dos jobs cloud em CI   | Orçamento pessoal                 | Jobs cloud opcionais e manuais; local é o caminho padrão                            |

## 14. Fora de escopo do MVP

Deliberadamente excluídos, para proteger a tese: memória longa de agentes; RAG; roteamento de modelos; FinOps; orquestração multi-agente; dashboard enterprise; detecção genérica por deep learning; identidade própria (IdP); blockchain; LLM-as-judge; inferência causal completa (Shapley sobre nós, confundidores); inventário de agentes; quarentena autônoma; substituição de IAM. Cada item só entra depois que a propriedade No Double-Spend e a curva coordenação × autonomia estiverem medidas e publicadas.

## 15. Decisão de IP e open source

- Publicação do repositório só após o Sprint 0 e a decisão formal do Sprint 6. Publicar antes de depositar consome o período de graça (12 meses no Brasil e nos EUA; inexistente na Europa).
- Candidatos a claim, condicionados a prior art: (i) emissão de credencial efêmera consumível com residual e certificado de contenção verificável offline; (ii) protocolo híbrido partição/CAS para preservação de invariantes globais entre agentes concorrentes; (iii) vinculação de autorização a hash de precondições de estado com reverificação limitada.
- Software em si não é patenteável no Brasil (Lei 9.279/96, art. 10) nem sob o teste Alice nos EUA; os claims devem descrever mecanismo técnico com efeito mensurável (double-spend zero, latência, overhead). Consultar profissional de PI antes de qualquer depósito.
- Candidatos a OSS independentemente da decisão de IP: ASIR schema, adaptadores, DSL de invariantes, VERITAS-Bench.
- Nome: evitar 'Aegis' (já usado na literatura de 2025–2026); confirmar disponibilidade de 'VERITAS' em registros de marca e em PyPI antes do Sprint 1.

## Referências

CUTLER, J. W. et al. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proc. ACM Program. Lang.* (OOPSLA), 2024. arXiv:2403.04651.

AMAZON WEB SERVICES. Enforce least-privilege authorization in multi-agent AI chains using Cedar. AWS Security Blog, jul. 2026.

STRANDS AGENTS. Cedar Authorization — intervention handler documentation, 2026.

NIST. AI Agent Standards Initiative; Concept Paper on Software and AI Agent Identity and Authorization; Security Considerations for AI Agents, 2026.

NECULA, G. C. Proof-Carrying Code. *POPL*, 1997.

VOVK, V.; GAMMERMAN, A.; SHAFER, G. *Algorithmic Learning in a Random World*. Springer, 2005.

BARBER, R. F. et al. Conformal prediction beyond exchangeability. *Annals of Statistics*, 2023.

ANGELOPOULOS, A. N. et al. Conformal Risk Control. *ICLR*, 2024.

CLARKE, E.; BIERE, A.; RAIMI, R.; ZHU, Y. Bounded Model Checking Using Satisfiability Solving. *Formal Methods in System Design*, 2001.

HERLIHY, M.; WING, J. Linearizability: A Correctness Condition for Concurrent Objects. *ACM TOPLAS*, 1990.

ONGARO, D.; OUSTERHOUT, J. In Search of an Understandable Consensus Algorithm (Raft). *USENIX ATC*, 2014.

IETF. JSON Canonicalization Scheme (JCS). RFC 8785, 2020.

OWASP. Top 10 for Agentic Applications, 2026.

BRASIL. Lei nº 9.279, de 14 de maio de 1996 (Lei da Propriedade Industrial), art. 10.

Crizan Belem Ribeiro — VERITAS: Plano End-to-End do MVP — v1.0, agosto de 2026.